

Stock watchlist and price alerts

Assignment 1 report | Issam Arida | 4 October 2026

1. What I built and who it is for

I built a small web app for people who invest a bit of their own money in US stocks. They usually have a job or are studying, so they can't sit and watch a chart all day. They want a list of the stocks they follow with a note on why, and they want to know when a price crosses a level they care about.

I pictured a small pilot of up to 50 investors on one server, each with up to 30 stocks and 100 alert rules. Those numbers became the default limits in the code. Each person gets a private account because I didn't want one user reading or changing another user's notes.

The app has two feature domains. The watchlist holds the stocks you follow with a name and a note. Alerts holds one-shot price rules ("tell me when AAPL goes above 200") plus a history of every rule that fired. Both domains read and write SQLite. Prices come from Finnhub, a public market data API. Login is support code around the two domains and I don't count it as a third domain.

Everything runs as one Python process with one SQLite file under DATA_DIR. Inside that process a background thread checks the alert rules every 5 seconds, and two more threads keep a live WebSocket connection to Finnhub open.

2. How I worked: SDLC and SMART goals

I used an iterative and incremental model. I built the app in thin working slices: first the scaffold and the deployment contract, then the watchlist, then alerts, then the price adapters and the poller, then tests and docs. Every slice ran end to end before I started the next one. I picked this because I had about a week and one person. Waterfall would have locked my domain split in on day one, and I changed my mind about it more than once. Scrum is built for teams with sprints and standups, which is a lot of ceremony for a solo project.

My SMART goals and what actually happened:

- **Both domains working on SQLite by 1 October.** Done early. Watchlist landed on 28 September and alerts on 30 September.
- **At least 70% coverage on core logic by 1 October.** Done. The first measured run was already at 100% of the scope I defined in .coveragerc.
- **A fresh clone starts with python app.py and an empty DATA_DIR in under five seconds, by 2 October.** Done. A contract test starts the real entry point on a random port and waits for /health to return 200.
- **Five ADR entries across at least three commit dates by 2 October.** Done. The entries landed on 28 September, 30 September and 1 October.

Where I didn't follow the plan well was pacing. I created the repo on 23 September but only started real work on 28 September, so the week got squeezed. Ten commits landed on 30 September alone. A few of my early commit messages are lazy ("update", "rcorrected", "init") and I know those don't count as meaningful under the brief.

The plan also changed late, which is what an iterative model is meant to handle. On 3 October I reviewed the app against my own audience and realised one shared anonymous watchlist made no sense for real investors. I added private accounts and fixed a race where a deleted rule could fire its replacement. On the same day I dropped the fake demo prices and moved to real Finnhub data. On 4 October I turned the watchlist page into a live dashboard. I recorded each of these as a dated revision inside the existing ADRs instead of adding new entries, because the brief asks for exactly five.

In total there are 31 commits over 8 calendar days. The busiest day has 10 commits, which is 32% of the total and under the 40% limit. I used feature/* branches merged into develop and then main for the bigger changes.

3. Architecture

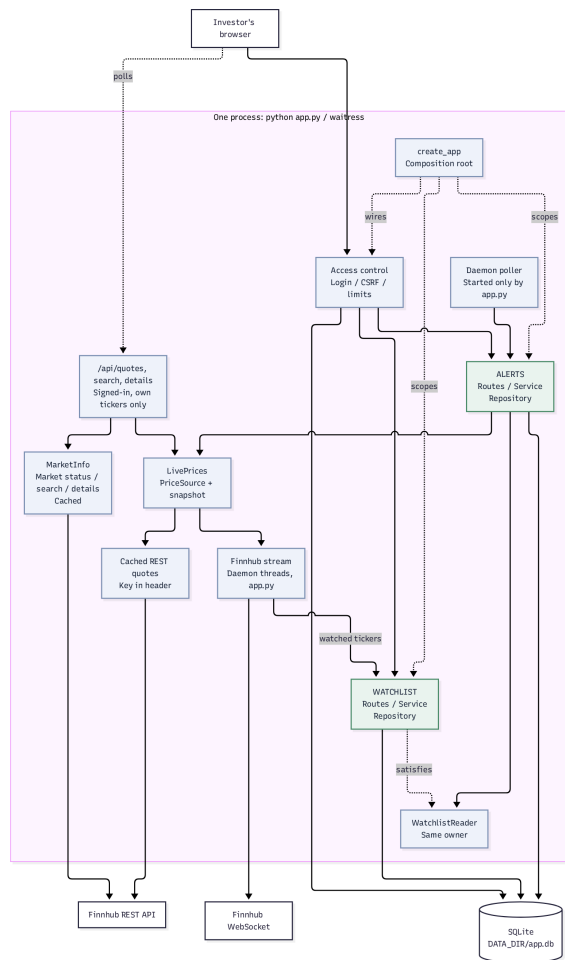


Figure 1. The whole app runs in one process. Green boxes are the two feature domains. Solid arrows show calls and data, dotted arrows show wiring.

`python app.py` is the only start command. It reads settings from environment variables, calls `create_app`, starts the Finnhub stream and the alert poller, then hands the app to waitress on `HOST:PORT`. `create_app` builds everything and never starts a thread, so tests can build the app without touching the network.

Both domains use the same layout. Routes read the form and call the service. The service checks the input and applies the rules. The repository runs plain SQL with one short-lived connection per call, so threads never share a connection.

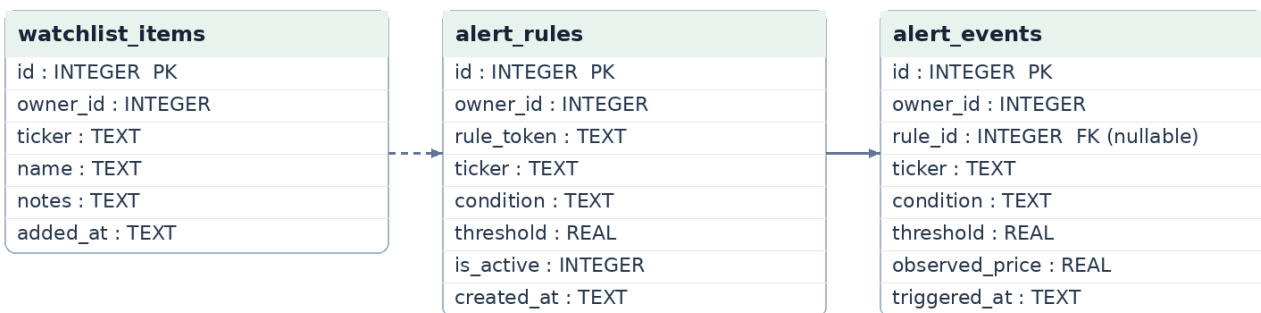
The seam between the domains is `app/ports.py`. Alerts needs two answers from outside: is this ticker on the user's watchlist, and what does it cost right now. Those are two small interfaces, `WatchlistReader` and `PriceSource`. Alerts imports only those and never imports the watchlist or market code. `create_app` is the only file that plugs the real classes in. A test reads every import under `app/` and fails the build if anyone breaks that rule.

That is where I'd cut the app into services in Assignment 2. Alerts would take its two tables and the poller with it, and `WatchlistReader` would become a small HTTP client that calls the watchlist service. I'd still have to pass the user's identity across and handle the network failing. The seam only takes care of the code side.

Prices come from FintHub in two ways. One WebSocket connection from the server streams live trades for every ticker that any user watches. When there's no recent trade, for example at the weekend, a cached REST call returns the latest quote. The browser never talks to FintHub and never sees the API key. The pages ask this server for prices once a second at `/api/quotes`, which only returns the signed-in user's own tickers.

The dashboard shows each stock's live price, today's change, a 20-minute trend line built from streamed trades, the day range and the 52-week range. FintHub's price history endpoint is paid only, so I draw the trend from the trades the server already receives. When the poller fires a rule, the page shows a notice within a second. I used Jinja templates with one inline script and no frontend build, because two pages don't need React.

4. Database



Logical link: same owner + ticker. No cross-domain foreign key.

Event to rule: 0 or 1 rule. Rule to events: 0 or many in the schema; the application fires each rule once. Deletion sets `rule_id` to NULL.



UNIQUE(owner_id, ticker) in `watchlist_items`. Account ownership is enforced by scoped queries, without foreign keys to accounts.

PK = primary key FK = foreign key Supporting tables have no domain joins.

Figure 2. The five tables in `DATA_DIR/app.db`. The dotted line between watchlist and alert rules is a link by ticker text only, with no foreign key.

The watchlist owns `watchlist_items`. Alerts owns `alert_rules` and `alert_events`. The other two tables, `accounts` and `rate_limits`, belong to login and throttling.

The decision I care about most (ADR-3) is how the two domains relate. `alert_rules.ticker` is plain text with no foreign key to `watchlist_items`, and no query ever joins across the two domains. If alerts moves to its own service with its own database later, nothing in its schema points at a table it can't reach. The cost is that a rule can outlive its stock. If you remove AAPL from your watchlist, its rules stay in the table as dormant and come back to life when you add AAPL again.

`alert_events` stores a copy of the ticker, condition, threshold, observed price and time when a rule fires. That way the history still reads correctly after you delete the rule, and `rule_id` just becomes NULL. Keeping only a "last fired" column on the rule would have lost the history on delete.

Rules are one-shot. `fire_rule` takes a write lock with `BEGIN IMMEDIATE`, switches the rule off and writes the event in one transaction. If two checks run at once, only one of them wins. Each rule also has a

random `rule_token`, so a check that started before you deleted a rule can't fire a new rule that reused its ID.

Every business table has an `owner_id`, and every query filters on the signed-in user's ID. The tables are created at startup with `CREATE TABLE IF NOT EXISTS`, and an older single-user database is upgraded automatically with its rows parked under owner 0, which nobody can log in as. SQLite stays in its default journal mode.

5. Testing

Tests target the core logic of both domains: `is_triggered`, `AlertService.evaluate_all`, `fire_rule` and the watchlist validation. They use fake `PriceSource` and `WatchlistReader` classes, so they're fast and never touch the network. Route files and `create_app` are left out of coverage because they're thin glue. One end-to-end test drives the full flow from adding a stock to a fired alert through the Flask test client.

The command is `pytest --cov=app --cov-report=term-missing`. On 4 October, 240 tests passed with 100% statement coverage over 868 statements. Coverage below 70% fails the run.

The weak spot is that the fakes would not notice if Finnhub changed its response format. I checked the real API by hand with my own key, and the tests cover malformed answers, but there is no automated test against the live service.

6. Deployment contract and setup

I went through section 7 of the brief point by point:

- One process, started with `python app.py`.
- Binds `0.0.0.0` by default and reads the port from `PORT`, default 8080.
- Nothing interactive at startup. Tables are created automatically.
- SQLite lives at `$DATA_DIR/app.db`, default `./data/app.db`.
- The only outside dependency is Finnhub. Without a key the app still starts and shows prices as unavailable.
- `requirements.txt` is the only manifest, with seven packages. There's no Dockerfile, compose file, CI workflow or IaC.
- Every setting is an environment variable with a default in code. `app.py` will read a git-ignored `.env` file if one exists, but nothing needs it.
- The contract test checks that the app is healthy within five seconds of starting.

To run it:

```
uv venv
source .venv/bin/activate.fish
uv pip install -r requirements.txt
set -x PRICE_API_KEY your-finnhub-key
python app.py
```

Then open `http://localhost:8080`, create an account and add a ticker. The README has the full list of settings and the coverage command.

For Assignment 2 the container needs a persistent volume for `DATA_DIR`, one replica, HTTPS with `SESSION_COOKIE_SECURE=true` and a fixed `SECRET_KEY`. Without a fixed key everyone has to log in again after a restart.

7. What I chose not to build

I left out notifications by email or push and I left out trading (ADR-5). Both need an outside service or a lot of extra code I'd have to secure and explain, and the core use case works without them. Fired alerts show up on screen and in the history.

8. AI disclosure

I acknowledge the use of Claude Code (Claude Opus 5.5) and Codex to plan and write most of the code and tests with drafts of the documentation, and to review the project against the assignment brief. The prompts used include building the watchlist and alerts domains behind small interfaces, adding the poller and the price cache, writing tests to reach the coverage target, adding private accounts, replacing demo prices with live Finnhub data and turning the watchlist into a real-time dashboard. The output of these prompts was used to create and revise the code in `app/`, the tests, the README, the ADR entries, the diagrams and this report, after I read and checked it. Every interaction is logged in `AI_USAGE.md`, and its last column is where I write in my own words how the accepted code works.